PrintOnline.et v2.0 — MVP Task Workflow Breakdown

> **Version:** 2.0.0
> **Date:** 2026-02-26
> **Status:** READY FOR IMPLEMENTATION
> **Launch Window:** 7 hours from 16:30 EAT
> **Deadline:** ~23:30 EAT Today

Overview

This document breaks v2.0 into **7 phases** with estimated hours and granular tasks. Phases are ordered by dependency — each phase builds on the previous. Within each phase, tasks are ordered by dependency.

> [!IMPORTANT] **Phase 0-2** must complete first (MVP pipeline). **Phases 3-6** can be partially parallelized. **Phase 7** is the final deployment step.

```
Phase 0: Infrastructure Setup ──────────────── (1h)
    │
    ▼
Phase 1: Database Schema & Data ─────────────── (2h)
    │
    ▼
Phase 2: Core Backend Integration ──────────── (3h)
    │
    ├──────────────────┬──────────────────┐
    ▼                  ▼                  ▼
Phase 3: Auth       Phase 4: CMS        Phase 5: Frontend
  (2.5h)              (4h)               Modernization (3h)
    │                  │                  │
    └──────────────────┼──────────────────┘
                       │
                       ▼
               Phase 6: Order Flow + Email ─── (2.5h)
                       │
                       ▼
               Phase 7: Deployment ──────────── (1h)
```

Total Estimated: ~19 hours
Today's Target: Phases 0-2 + critical items from 3-5 + deploy

---

Phase 0: Infrastructure Setup (1h)

0.1 — Environment Variables

Task	Details
Create `.env.local`	Supabase URL, anon key, service role key, better-auth secret

Task	Details
Create `.env.example`	Template without secrets
Remove `ignoreBuildErrors`	Delete from [next.config.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/next.config.ts)

Files:

- `[NEW] .env.local`
- `[NEW] .env.example`
- `[MODIFY]` [next.config.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/next.config.ts) — remove `ignoreBuildErrors: true`

0.2 — Supabase Client Libraries

Task	Details
Browser client	Public anon key, for client components
Server client	For RSC + API routes (with cookies)
Admin client	Service role key, for CMS operations

Files:

- `[NEW] lib/supabase/client.ts`
- `[NEW] lib/supabase/server.ts`
- `[NEW] lib/supabase/admin.ts`

0.3 — Install Dependencies

```
npm install @supabase/supabase-js @supabase/ssr nodemailer
npm install -D @types/nodemailer supabase
```

Acceptance:

- `npx supabase --version` returns version
- Supabase clients importable
- `.env.local` loaded in dev server

Phase 1: Database Schema & Data (2h)

1.1 — SQL Migrations

Write and execute all 8 migration files.

Migration	Tables/Objects
001	`categories`

Migration	Tables/Objects
002	`products`
003	`product_images`
004	`product_options`, `product_option_values`
005	`customer_profiles`
006	`orders`, `order_items`, order number sequence + trigger
007	RLS policies
008	`updated_at` triggers

Files:

- `[NEW] supabase/migrations/001_categories.sql` through `008_functions.sql`

Execution: Run via Supabase Dashboard SQL Editor or `supabase db push`.

1.2 — Seed Product Data

Transform extracted PDF catalog into SQL INSERT statements.

Strategy:

1. Create categories (Digital Paper Prints, Promotional Items, Vinyl Prints & Wraps, etc.)
2. Create all 15 products with `base_price` in ETB
3. Create product options per product (Size, Paper, Lamination, etc.)
4. Create option values with pricing data

Files:

- `[NEW] supabase/seed/categories.sql`
- `[NEW] supabase/seed/products.sql`
- `[NEW] supabase/seed/product_options.sql`

1.3 — Generate TypeScript Types

```
npx supabase gen types typescript --project-id <id> > types/database.ts
```

Files:

- `[NEW] types/database.ts` (auto-generated)
- `[NEW] types/index.ts` (convenience re-exports)

1.4 — Product Image Upload

Upload provided images to Supabase Storage. Use name-matching function.

Files:

- `[NEW] lib/image-mapper.ts` — smart name-to-product matching
- Script to bulk-upload to `product-images` bucket

Acceptance:

- All tables exist in Supabase with correct columns
- 15+ products seeded with options and values
- `types/database.ts` generated with all table types
- Product images accessible via public URLs

Phase 2: Core Backend Integration (3h)

2.1 — Currency Utility

Task	Details
ETB formatter	`formatETB(amount)` → "ETB 350.00"
PriceDisplay component	`<PriceDisplay amount={price} />`

Files:

- `[NEW] lib/currency.ts`
- `[NEW] components/shared/PriceDisplay.tsx`

2.2 — TanStack Query Hooks

Build typed hooks for all entities using the generated Supabase types.

Hook	Purpose	Query Key
`useProducts`	List products with filters	`['products', filters]`
`useProduct`	Single product by slug	`['products', slug]`
`useProductOptions`	Options + values for product	`['product-options', productId]`
`useCategories`	All active categories	`['categories']`
`useOrders`	User's orders	`['orders', userId]`
`useOrder`	Single order by ID	`['orders', orderId]`
`useCustomers`	CMS: all customers	`['customers']`
`useSearch`	Full-text search	`['search', query]`

Files:

- `[NEW] hooks/data/useProducts.ts` (replace existing)
- `[NEW] hooks/data/useProduct.ts`
- `[NEW] hooks/data/useCategories.ts`
- `[NEW] hooks/data/useOrders.ts`
- `[NEW] hooks/data/useSearch.ts`

- [NEW] hooks/data/useCustomers.ts
- [NEW] hooks/data/useProductOptions.ts

2.3 — Zod Validation Schemas

Derive Zod schemas from generated types for all inputs.

Files:

- [NEW] lib/validations/product.ts
- [NEW] lib/validations/order.ts
- [NEW] lib/validations/customer.ts
- [NEW] lib/validations/auth.ts

2.4 — Replace Hardcoded Product Data

Task	Details
Delete [lib/products.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/lib/products.ts)	Remove all 756 lines of mock data
Update product page	Fetch from Supabase via `useProduct(slug)`
Update category pages	Fetch from Supabase via `useProducts({ category })`
Update home page	Fetch featured/popular from Supabase
Update all-products page	Fetch all from Supabase
Unify Product type	Single [Product](file:///c:/Users/kidus/Documents/Projects/printonline-et/components/product/ProductDetailPage.tsx#36-58) type from `types/database.ts`

Files:

- [DELETE] [lib/products.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/lib/products.ts)
- [MODIFY] app/(storefront)/products/[slug]/page.tsx
- [MODIFY] app/(storefront)/page.tsx
- [MODIFY] app/(storefront)/all-products/page.tsx
- [DELETE] [types/product.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/types/product.ts) (replaced by `types/database.ts`)
- [MODIFY] [context/CartContext.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/context/CartContext.tsx) — align types

2.5 — Error Boundaries & Suspense

Task	Details
Global error boundary	app/error.tsx

Task	Details
Route-level boundaries	One per route group
Loading states	`loading.tsx` files for key routes
ErrorFallback component	Reusable error UI

Files:

- `[NEW] app/error.tsx`
- `[NEW] app/(storefront)/error.tsx`
- `[NEW] app/(storefront)/products/[slug]/error.tsx`
- `[NEW] app/(storefront)/products/[slug]/loading.tsx`
- `[NEW] app/(cms)/error.tsx`
- `[NEW] components/shared/ErrorFallback.tsx`
- `[NEW] components/shared/LoadingState.tsx`

Acceptance:

- [lib/products.ts](file:///c:/Users/kidus/Documents/Projects/printonline-et/lib/products.ts) deleted — zero hardcoded product data
- All pages fetch from Supabase
- All prices display in ETB
- Error boundaries catch and display errors gracefully
- Loading states shown during data fetch

Phase 3: Authentication (2.5h)

3.1 — better-auth Configuration

Task	Details
Auth server config	Email/password, custom fields (phone, TIN, company)
Auth client	Client-side auth helpers
API route handler	`app/api/auth/[...all]/route.ts`
Middleware	Protect `/account/*` and `/cms/*` routes

Files:

- `[NEW] lib/auth.ts` — server-side config
- `[NEW] lib/auth-client.ts` — client-side helpers
- `[NEW] app/api/auth/[...all]/route.ts`
- `[NEW] middleware.ts` — route protection

3.2 — Auth Pages

Page	Features

Page	Features
Login	Email/password, redirect support
Register	Name, email, password, phone, TIN (optional), company (optional)
Forgot Password	Email reset flow

Files:

- `[NEW] app/(auth)/login/page.tsx`
- `[NEW] app/(auth)/register/page.tsx`
- `[NEW] app/(auth)/forgot-password/page.tsx`
- `[NEW] app/(auth)/layout.tsx`

3.3 — Customer Profile

Task	Details
Profile creation	Auto-create `customer_profiles` on registration
Profile management	Edit name, phone, TIN, address
Account dashboard	Profile + order summary

Files:

- `[NEW] app/(account)/layout.tsx`
- `[NEW] app/(account)/account/page.tsx`
- `[MODIFY] app/(account)/orders/page.tsx` (refactor existing)

Acceptance:

- Registration creates user + customer_profiles row
- Login works with redirect
- Protected routes redirect to login
- Profile editable with Zod validation
- TIN and company_name fields available

Phase 4: CMS Admin Panel (4h) — P0

4.1 — CMS Layout & Navigation

Task	Details
CMS sidebar	ERP-style navigation (Products, Orders, Customers, Categories)
CMS header	Breadcrumb, user info, logout
CMS dashboard	Quick stats (total orders, products, customers)

Files:

- `[NEW] app/(cms)/layout.tsx`

- `[NEW] app/(cms)/cms/page.tsx` — dashboard
- `[NEW] components/cms/layout/CMSSidebar.tsx`
- `[NEW] components/cms/layout/CMSHeader.tsx`

4.2 — Product Management (CMS)

Feature	Details
Product list	Searchable, filterable, sortable table
Create product	Full form: all fields + options + images
Edit product	Same form, pre-populated
Delete product	Confirm dialog
Option editor	Add/edit/remove product options and values
Image manager	Upload, reorder, set primary, delete
Stock status	Toggle in_stock/low_stock/out_of_stock/made_to_order

Files:

- `[NEW] app/(cms)/cms/products/page.tsx`
- `[NEW] app/(cms)/cms/products/new/page.tsx`
- `[NEW] app/(cms)/cms/products/[id]/page.tsx`
- `[NEW] app/(cms)/cms/products/[id]/edit/page.tsx`
- `[NEW] components/cms/products/ProductForm.tsx`
- `[NEW] components/cms/products/ProductList.tsx`
- `[NEW] components/cms/products/ProductOptionEditor.tsx`
- `[NEW] components/cms/products/ProductImageManager.tsx`

4.3 — Order Management (CMS)

Feature	Details
Order list	Filterable by status, searchable, sorted by date
Order detail	Full order information, item list, customer info
Status editor	Dropdown to change status → triggers email + status_history entry
Internal notes	Admin-only notes field

Files:

- `[NEW] app/(cms)/cms/orders/page.tsx`
- `[NEW] app/(cms)/cms/orders/[id]/page.tsx`
- `[NEW] components/cms/orders/OrderList.tsx`
- `[NEW] components/cms/orders/OrderDetail.tsx`
- `[NEW] components/cms/orders/OrderStatusEditor.tsx`

4.4 — Category & Customer Management (CMS)

Files:

- `[NEW] app/(cms)/cms/categories/page.tsx`
- `[NEW] app/(cms)/cms/categories/[id]/page.tsx`
- `[NEW] app/(cms)/cms/customers/page.tsx`
- `[NEW] app/(cms)/cms/customers/[id]/page.tsx`

4.5 — CMS Shared Components

Files:

- `[NEW] components/cms/shared/CMSPageHeader.tsx`
- `[NEW] components/cms/shared/CMSDataTable.tsx`
- `[NEW] components/cms/shared/CMSConfirmDialog.tsx`
- `[NEW] components/cms/shared/CMSEmptyState.tsx`
- `[NEW] components/cms/shared/CMSStatsCard.tsx`

Acceptance:

- Full product CRUD with option/value/image management
- Order list with status filters
- Status change creates status_history entry
- CMS UI matches ERP aesthetic
- Admin-only access (middleware-protected)

Phase 5: Frontend Modernization (3h)

5.1 — Component Decomposition (P0)

Component	Current	Target
[ProductDetailPage.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/components/product/ProductDetailPage.tsx)	935 lines	8 files, ~100-200 lines each
[Header.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/components/Header.tsx)	247 lines	Header + CategoryNav + CartDrawer + SearchBar
[Footer.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/components/Footer.tsx)	245 lines	Footer (OK as-is, audit only)
[checkout/page.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/app/checkout/page.tsx)	493 lines	3-4 step components
[order-summary/page.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/app/order-summary/page.tsx)	630 lines	4-5 components

Component	Current	Target
[order-confirmation/page.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/app/order-confirmation/page.tsx)	396 lines	3 components
[cart/page.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/app/cart/page.tsx)	234 lines	CartPage + CartItemRow

Product Detail Decomposition Files:

- `[MODIFY]` [components/product/ProductDetailPage.tsx](file:///c:/Users/kidus/Documents/Projects/printonline-et/components/product/ProductDetailPage.tsx) — orchestrator only (~150 lines)
- `[NEW] components/product/ProductGallery.tsx`
- `[NEW] components/product/ProductInfo.tsx`
- `[NEW] components/product/ProductForm.tsx`
- `[NEW] components/product/ProductActions.tsx`
- `[NEW] components/product/ProductTabs.tsx`
- `[NEW] components/product/ProductBreadcrumb.tsx`
- `[NEW] components/product/RelatedProducts.tsx`

5.2 — Shared Components

Files:

- `[NEW] components/shared/ProductCard.tsx`
- `[NEW] components/shared/ProductGrid.tsx`
- `[NEW] components/shared/PriceDisplay.tsx` (from 2.1)
- `[NEW] components/shared/QuantitySelector.tsx`
- `[NEW] components/shared/SearchBar.tsx`
- `[NEW] components/shared/FileUpload.tsx`
- `[NEW] components/shared/OrderStatusBadge.tsx`
- `[NEW] components/shared/CategoryNav.tsx`
- `[NEW] components/shared/CartDrawer.tsx`
- `[NEW] components/shared/EmptyState.tsx`

5.3 — Catch-All Category Routes

Task	Details
Create catch-all	`categories/[slug]/page.tsx`
Delete old routes	6 separate category page directories
Update navigation	All links point to `/categories/{slug}`

Files:

- `[NEW] app/(storefront)/categories/[slug]/page.tsx`
- `[DELETE] app/digital-paper-prints/`
- `[DELETE] app/signage-solutions/`

- `[DELETE] app/flex-banners/`
- `[DELETE] app/vinyl-prints/`
- `[DELETE] app/fabric-prints/`
- `[DELETE] app/promotional-items/`

5.4 — Cart Slide-Over Drawer

Files:

- `[NEW] components/shared/CartDrawer.tsx`
- `[MODIFY] components/Header.tsx` — integrate CartDrawer trigger
- `[MODIFY] context/CartContext.tsx` — add DB sync for auth users

5.5 — Dark Mode Audit

Scan and fix all hardcoded colors. Target: every component uses theme tokens exclusively.

5.6 — Currency Conversion

Replace all `$` and USD references with ETB formatting via `<PriceDisplay>`.

Acceptance:

- No component file exceeds ~200 lines
- All shared components in `components/shared/`
- One catch-all category route
- Cart drawer slides over from right
- No hardcoded colors in any component
- All prices in ETB

Phase 6: Order Flow + Email (2.5h)

6.1 — Order API Routes

Files:

- `[NEW] app/api/orders/route.ts` — POST (create order)
- `[NEW] app/api/orders/[id]/route.ts` — GET, PUT (status update)
- `[NEW] app/api/upload/route.ts` — POST (design file upload)

6.2 — Order Summary Refactor

Task	Details
Add T&C section	Checkbox + modal/expandable content
Generate T&C content	Standard e-commerce T&C for Ethiopian printing services
Special instructions	Textarea field
Design file upload	Connect to Supabase Storage
Price breakdown	Subtotal, delivery fee (if applicable), total

Files:

- `[MODIFY] app/(storefront)/order-summary/page.tsx`
- `[NEW] components/checkout/TermsAndConditions.tsx`
- `[NEW] content/terms-and-conditions.md`

6.3 — Order Confirmation

Task	Details
Persist order to DB	POST to `/api/orders`
Display order number	`POL-2026-XXXXX` format
Show order status	Initial "Pending"
Account link	"View in your account"

6.4 — Email Integration

Task	Details
Nodemailer transport	SMTP config from env
Order notification email	Rich HTML to order@printonline.et
Customer confirmation	Rich HTML to customer email
Status update email	Template for status changes

Files:

- `[NEW] lib/email.ts`
- `[NEW] lib/email-template.ts`
- `[NEW] app/api/send-order-email/route.ts`

Acceptance:

- Orders persist to Supabase
- T&C checkbox required before placing order
- Email sent to order@printonline.et on order creation
- Email sent to customer on status change
- Design files uploaded to Supabase Storage

Phase 7: Deployment (1h)

7.1 — Vercel Configuration

Task	Details
Connect repo to Vercel	GitHub integration
Set environment variables	Supabase keys, SMTP config, better-auth secret

Task	Details
Configure domain	printonline.et DNS settings
Build verification	`next build` passes with zero errors

7.2 — Production Checklist

```
[ ] .env variables set in Vercel
[ ] Build succeeds with no errors
[ ] All pages load without errors
[ ] Product data displays from Supabase
[ ] Auth flow works (register → login → profile)
[ ] CMS accessible and functional
[ ] Orders can be placed
[ ] Email notifications work
[ ] Dark mode functions across all pages
[ ] Mobile responsiveness verified
[ ] Error boundaries display correctly
```

Today's Execution Plan (7 Hours)

Given the 7-hour window, here is the **realistic prioritized schedule:**

Time	Phase	Tasks	Priority
16:30-17:00	Phase 0	Env vars, Supabase clients, deps	●
17:00-18:30	Phase 1	SQL migrations, seed data, type gen	●
18:30-20:30	Phase 2	Query hooks, kill mock data, error boundaries	●
20:30-21:30	Phase 5 (partial)	ProductDetailPage decomposition, PriceDisplay, ETB	●
21:30-22:30	Phase 3 (partial)	better-auth setup, login/register	◐
22:30-23:00	Phase 7	Vercel deploy, domain config	●
23:00-23:30	Buffer	Fix issues, verify production	◐

> [!WARNING] **Phases 4 (CMS) and 6 (Email)** will extend into tomorrow's sprint. The CMS is P0 but requires 4+ hours of focused work. Today's goal is to have the **storefront functional with real data and deployed**.

Verification Plan

Automated Checks

```
# Type check (must pass with zero errors)
npx tsc --noEmit
```

```
# Build check (must pass)
npm run build

# Lint check
npm run lint
```

Manual Verification

1. **Home page** — Products load from Supabase, prices in ETB, images display
2. **Product detail** — Options from DB, form validation works, add to cart functional
3. **Category pages** — Dynamic route loads correct products
4. **Cart drawer** — Opens from header, shows items with options
5. **Auth flow** — Register → login → profile visible
6. **CMS** — Products list → create → edit → status change
7. **Order flow** — Cart → checkout → order summary → T&C → confirm → email received
8. **Dark mode** — Toggle and verify every page
9. **Mobile** — Test all pages at 375px width
10. **Error states** — Disconnect internet, verify error boundaries

---

Ready for implementation. Awaiting final confirmation.